

From GitHub Action Workflow to GitHub App

Your current setup utilizes a GitHub Action workflow file (`.yaml`) that lives within a single repository. When a PR is raised in *that* specific repository, the workflow triggers and runs your logic.

To apply this to **all** repositories under your username, you must move the trigger mechanism outside of the individual repos and into a centralized service that listens to events from your entire account. This service is a **GitHub App**.

Feature	GitHub Action Workflow	GitHub App
Scope	One repository (where the <code>.github/workflows</code> folder exists).	All, none, or selected repositories on an account/organization.
Trigger	Code runs directly within a virtual machine hosted by GitHub.	Sends a webhook (an HTTP request) to an external server when an event occurs.
Best For	CI/CD, simple automation, single-repo tasks.	Cross-repository agents, deep integrations, centralized services.

Steps to Implement Cross-Repository PR Review

Your existing code (the logic within `github_pr_reviewer`) is the core of your new GitHub App. You now need to set up the infrastructure to host this code and listen for GitHub events.

Step 1: Register Your GitHub App

1. **Go to GitHub Settings:** Navigate to your profile settings, then select **Developer settings** in the sidebar, and finally **GitHub Apps**.
2. **Register a New GitHub App:** Click "New GitHub App."
3. **Configuration:**
 - **Homepage URL:** Enter your repository URL (e.g., `https://github.com/nehaduggirala/Github-issue-resolver`).
 - **Webhook URL:** This is the most crucial part. This is the publicly accessible URL where your existing PR reviewer code will be running and listening for events. You will need to deploy your code to a hosting service (like AWS Lambda, Google Cloud Functions, Vercel, or an internal server) to get this URL.
 - **Webhook Secret (Recommended):** Generate a strong secret string. This helps verify that the incoming webhook requests are truly coming from GitHub.

Step 2: Configure Permissions and Events

This tells GitHub what data the App needs access to and what events it should notify you about.

1. **Repository Permissions:**
 - **Pull requests:** Set this to **Read & Write** so your agent can fetch PR details and leave comments.

- **Contents:** Set this to **Read-only** if your agent needs to read file changes in the PR.
2. **Subscribe to Events:** This is where you tell the App *when* to send a webhook.
 - Check the **Pull request** event. This will send a webhook whenever a PR is opened, closed, synchronized, or a review is submitted.

Step 3: Install the GitHub App on All Repositories

This is the key step to achieving cross-repository access.

1. After registering the App, go to its public page (Settings > Developer settings > GitHub Apps, then select your App).
2. Click **Install App**.
3. Select the user or organization you want to install it for (likely your username).
4. **Crucially, when prompted, select the radio button for "All repositories" or choose "Select repositories" and manually grant access to all relevant ones.**

Step 4: Deploy and Adapt Your Reviewer Code

Your code needs to be slightly modified to listen for and process webhooks instead of running as a GitHub Action job.

1. **Hosting:** Deploy your existing code logic to a server or serverless platform (e.g., a simple Express server, a Flask app, or a cloud function).
2. **Webhook Listener:** Your hosted service must have an endpoint (matching the **Webhook URL** from Step 1) that accepts `POST` requests.
3. **Event Handling:** When a `POST` request arrives:
 - Verify the request signature using the **Webhook Secret**.
 - Read the request body. It will contain the PR payload.
 - Check the event type (from the `X-GitHub-Event` header) to ensure it's a `pull_request` event and the action is correct (e.g., `opened` or `synchronize`).
 - Your existing logic can then take the PR data and perform the review/comment.